

김철규

Unity / C# 게임 클라이언트 개발 포트폴리오

해당 포트폴리오 문서는 세부 코드 샘플 문서에서 다룬 핵심 구조를 프로젝트별로 요약한 포트폴리오입니다. 이 문서에서는 문제 정의, 설계 선택, 흐름 중심으로 정리했습니다.

공개 범위

- 실제 프로젝트 흐름과 클래스 역할은 유지
- 회사 내부 데이터, 세부 구현사항, 리소스 로딩, 서버 저장, 네이티브 연동 세부 구현은 제외
- 구조, 책임 분리, 실행 흐름 중심으로 축약

이력서	링크
경력기술서	링크
RFist 코드 샘플	링크 제작영상1 제작영상2
Slime Rush 코드 샘플	링크 제작영상1 제작영상2 제작영상3
Rfice 코드 샘플	링크 제작영상1 제작영상2 제작영상3

RFist - 네트워크 입력 동기화 코드 샘플

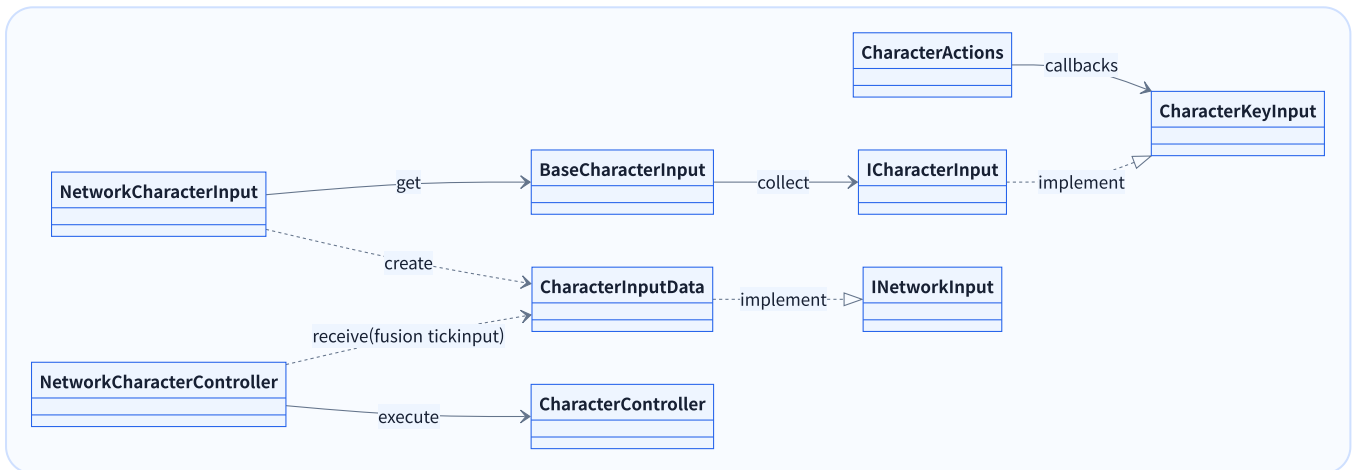
로컬 입력 캐시에서 네트워크 입력 수집, 캐릭터 액션 실행까지

제작 중점

- Input System callback 입력을 Fusion tick 입력 수집 시점에 안정적으로 전달해야 함
- 공격, 대시, 가드, 포커스 같은 입력은 프레임성 입력과 유지 입력을 구분해야 함
- 네트워크 입력이 실제 캐릭터 이동/전투 실행 계층과 명확히 연결되어야 함

설계 기준

- **CharacterInputData**: 네트워크 전송용 입력 데이터
- **CharacterKeyInput / ICharacterInput**: Input System callback을 게임 입력 인터페이스로 변환
- **BaseCharacterInput**: 로컬 입력 소스 수집
- **NetworkCharacterInput**: 권한 검증 및 입력셋
- **NetworkCharacterController**: 권한 검증 및 네트워크 전송
- **CharacterController**: 실제 캐릭터 이동/전투 동작



핵심 흐름

1. 입력 이벤트가 발생한 순간 바로 캐릭터를 움직이지 않고, 로컬 입력 구현체(CharacterKeyInput)가 입력 상태를 캐싱
2. Fusion 입력 수집 시점(NetworkCharacterInput)에는 캐시된 값을 읽어 네트워크 인풋 구조체(CharacterInputData)로 구성
3. 수신 측(NetworkCharacterController)에서는 네트워크 인풋 구조체를 캐릭터 이동/전투 실행 계층(CharacterController)으로 명령을 전달

RFist 코드 샘플

[링크](#)
[제작영상1](#)
[제작영상2](#)

Slime Rush - 마법 생성 및 자동 타겟팅

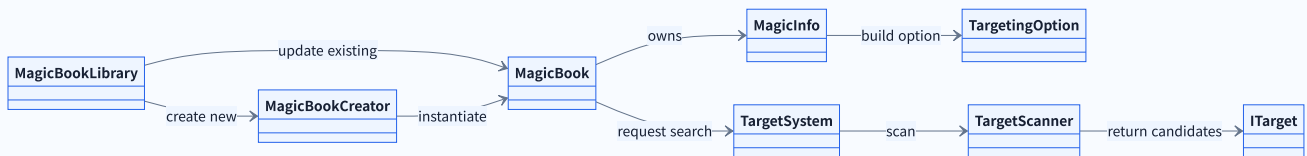
스킬 인스턴스 생성부터 타겟 검색까지

제작 중점

- 플레이어가 획득한 마법을 런타임 인스턴스로 생성하거나 갱신해야 함
- 마법별 타겟 타입, 타겟 수, 범위가 달라 자동 캐스팅 루틴이 복잡해질 수 있음
- 구체적인 비즈니스 로직보다 타겟을 찾는 구조가 명확해야 신규 마법 추가 비용을 줄일 수 있음

설계 기준

- **MagicBookLibrary**: 마법 할당 이벤트 수신과 생성/업데이트
- **MagicBookCreator**: MagicBook 생성 및 초기화
- **MagicBook**: 쿨타임, 타겟팅 옵션, 자동 검색 루틴 관리
- **TargetSystem**: 타겟 타입에 맞는 타겟/위치 검색 진입점
- **TargetScanner**: 거리, 랜덤, 개수 제한 기반 후보 탐색



핵심 흐름

1. 마법이 할당되면 마법 관리자(MagicBookLibrary) 기존 마법 인스턴스(MagicBook)를 갱신하거나 새 인스턴스를 생성
2. 마법 인스턴스는 마법 데이터(MagicInfo)에서 타겟팅 옵션(TargetingOption)을 만들고, 쿨타임 루틴 안에서 타겟 시스템(TargetSystem)에 타겟 검색을 요청
3. 실제 후보 탐색과 거리 검증은 타겟스캐너(TargetScanner) 쪽으로 분리해 자동 캐스팅 흐름과 검색 구조를 구분

Slime Rush 코드 샘플

[링크](#)
[제작영상1](#)
[제작영상2](#)
[제작영상3](#)

Rfice - 런타임 하우스징 에디터

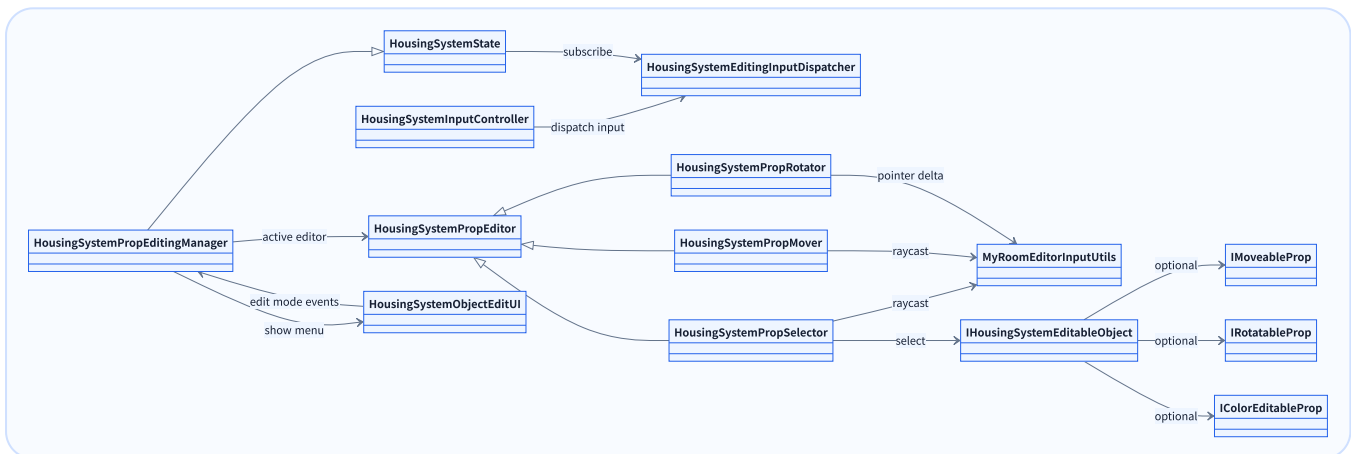
입력 디스패치부터 오브젝트 선택, 편집 모드 전환, 상태 적용까지

제작 중점

- 런타임 3D 공간에서 UI 클릭과 월드 오브젝트 클릭이 충돌하지 않아야 함
- 선택, 이동, 회전 편집이 같은 입력을 사용하지만 각기 다른 실행 흐름을 가져야 함
- 오브젝트마다 이동/회전/색상 편집 가능 여부가 달라 기능별 제약이 필요함

설계 기준

- **HousingSystemInputController**: InputAction 이벤트 수신
- **HousingSystemEditingInputDispatcher**: 편집 입력 이벤트 발행
- **HousingSystemState**: 편집상태에 맞는 입력 구독
- **HousingSystemPropEditingManager**: 선택 대상과 편집 모드 관리
- **HousingSystemPropEditor**: 선택/이동/회전 편집기의 공통 동작 정의



핵심 흐름

1. 입력은 핸들링(HousingSystemEditingInputDispatcher)을 통해 현재 편집 상태(HousingSystemState)로 전달
2. 편집매니저(HousingSystemPropEditingManager)가 선택된 오브젝트(IHousingSystemEditableObject)와 편집 모드(HousingSystemPropEditor)를 관리
3. 각각의 편집기(HousingSystemPropSelector, HousingSystemPropMover, HousingSystemPropSelector)가 기능을 담당하며, 이동/회전 완료 이벤트를 편집매니저가 구독해 배치 정보를 갱신

Rfice 코드 샘플

[링크](#)
[제작영상1](#)
[제작영상2](#)
[제작영상3](#)